

Topic Selected: 4 Club System

Team Members

Name	id
Ahmed Elsayed Allam	223102168
Basel Mohammed Ahmed	222100363
Mohamed Hassan Mohamed	224122908
Mostafa Ibrahim	222101590
Mohammed Abd El-Haliem	222101446

objectives

Club System: The required data is Club (Name, Branches, Manager, Location), Members (ID, Name, Phone, Number of children), and Sports (Name, ID, Number of teams).

- Design a java program that takes the required data from user.
- Sort Clubs by their names.
- Sort Members by their IDs.
- Sort Sports by their names.
- Search for club by its name.
- Search for member by its ID.
- Use three different sorting algorithms. Then calculate Big (O) of your program.

Program Behavior Overview

The system offers two user interfaces:

1. **Console** (`Main.java`) – A text-based menu.
2. **GUI** (`ClubManagementGUI.java`) – A Swing application.

Both interfaces share the same business logic through `ClubManagementService`, which handles all persistent data operations via an SQLite database. This guarantees identical behaviour regardless of the interface used.

Common Flow

- Data is represented by three immutable record types: `Club`, `Member`, `Sport`.
- `ClubManagementService` acts as the single point of access for reading and writing data; it internally delegates to `DatabaseHandler`.
- Sorting algorithms operate on the respective in-memory lists obtained from the service.
- Binary search always requires a pre-sorted list; the application enforces this by sorting automatically before every search.

Console Workflow

- The database is the authoritative source of data.
- After every add/remove operation, the in-memory lists are reloaded from the service.
- Sorting is performed on the in-memory lists; the database is not modified unless the user explicitly adds/removes entities.
- Statistics are computed using service-aggregated data.

GUI Workflow

- Data is loaded from the database via the service on startup and after every mutation.
- Sorting and searching are triggered by button clicks; results appear in tables or dialogs.
- The GUI includes additional controls for adding clubs, members, and sports, all of which are persisted immediately through the service.

Database Schema

The SQLite database `example.db` contains four core tables plus an association table.

PlantUML Diagram

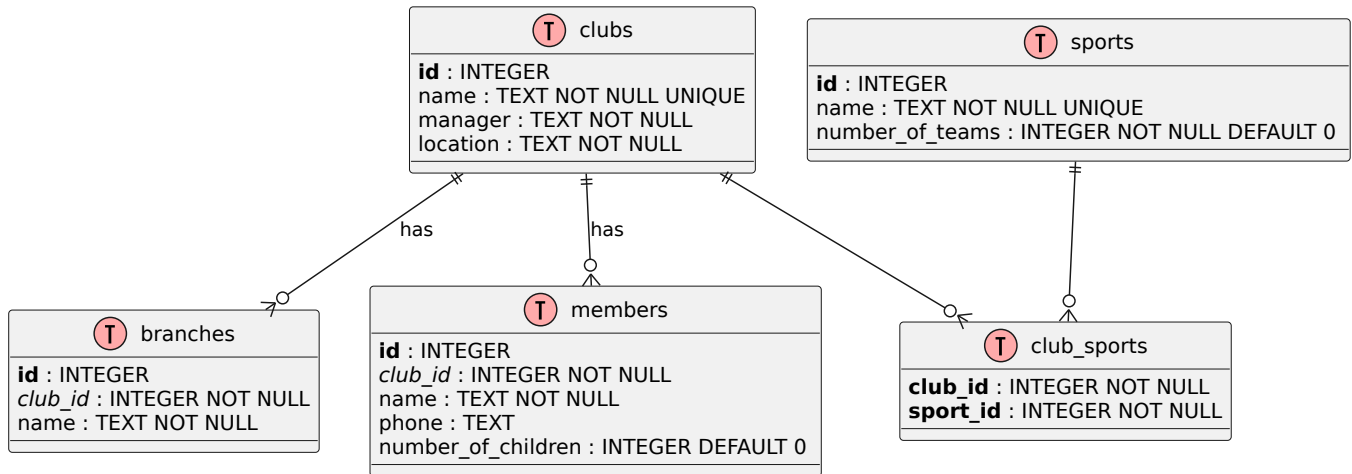


Figure 1: graph

Table Descriptions

clubs

- `id` – Auto-increment primary key.
- `name` – Unique club name (e.g., “Lions”).
- `manager`, `location` – Text fields.

branches

- Linked to `clubs` via `club_id` (foreign key with CASCADE delete).
- Multiple branches per club.

members

- `id` is a user-provided primary key (not auto-incremented).
- `club_id` foreign key to `clubs`.
- `number_of_children` defaults to 0.

sports

- `id` is user-provided.
- `name` unique.
- `number_of_teams` integer.

club_sports

- Many-to-many relationship between `clubs` and `sports`.
- Composite primary key (`club_id`, `sport_id`).
- No extra attributes.

SQL Files

- `schema.sql`: Contains `CREATE TABLE IF NOT EXISTS` statements for all tables.
- `data.sql`: Inserts sample clubs, branches, members, sports, and club-sport associations.

ClubSystem.java

Defines the core data structures using Java records and provides utility methods for initialisation and in-memory list manipulation.

Records

- `Club(String name, List<String> branches, String manager, String location, List<Member> members)` Immutable representation of a club, including its branches and members.
- `Member(int id, String name, String phone, int numberOfChildren)`
- `Sport(String name, int id, int numberOfTeams)`

Factory Methods

Method	Description
<code>initializeClubs()</code>	Creates five sample clubs (Lions, Tigers, Eagles, Sharks, Bears) with predefined members.
<code>initializeMembers()</code>	Returns a flat list of all sample members across clubs.
<code>initializeSports()</code>	Returns seven sports (Basketball, Football, Swimming, ...).

Utility Functions

Function	Complexity	Description
<code>displayClubs(List<Club>)</code>	$O(n)$	Prints formatted club details.
<code>displayMembers(List<Member>)</code>	$O(n)$	Prints member list.
<code>displaySports(List<Sport>)</code>	$O(n)$	Prints sport list.
<code>addMemberToClub(List<Club>, String clubName, Member)</code>	$O(c)$	Adds a member to a club (creates a new immutable <code>Club</code> instance).
<code>addClub(List<Club>, Club)</code>	$O(1)$	Appends a new club.
<code>addSport(List<Sport>, Sport)</code>	$O(1)$	Appends a new sport.
<code>removeMemberFromClub(List<Club>, String clubName, int memberId)</code>	$O(c + m)$	Removes a member from the matching club.
<code>findMemberById(List<Club>, int memberId)</code>	$O(c \times m)$	Searches all clubs for a member.
<code>getTotalMemberCount(List<Club>)</code>	$O(c)$	Sums member counts.
<code>getAllMembers(List<Club>)</code>	$O(c \times m)$	Flattens all club members into one list.

These functions are used by the CLI (`Main.java`) when the database is not involved. The GUI and `Main.java` with the database use the real persistence layer (`DatabaseHandler`).

Note: All mutation functions work with immutable records by creating new instances and replacing the original in the list.

DatabaseHandler.java

Singleton class that manages all SQLite operations for `example.db`. It automatically creates and populates the database on first run using `schema.sql` and `data.sql`.

Connection

- JDBC URL: `jdbc:sqlite:example.db`
- Auto-commit mode is enabled.

Initialisation

- On first start, the class checks if the `clubs` table exists.
- If not, it executes `schema.sql` and `data.sql` from the classpath (`/db/schema.sql`, `/db/data.sql`).

Public Methods

Method	Returns	Description
<code>getAllClubs()</code>	<code>List<Club></code>	Fetches clubs with branches and members.
<code>addClub(Club)</code>	<code>boolean</code>	Inserts club and its branches.
<code>getAllMembers()</code>	<code>List<Member></code>	Returns all members (regardless of club).
<code>addMemberToClub(String clubName, Member)</code>	<code>boolean</code>	Inserts a member linked to a club.
<code>removeMemberFromClub(String clubName, int memberId)</code>	<code>boolean</code>	Deletes the member.
<code>getAllSports()</code>	<code>List<Sport></code>	Retrieves all sports.
<code>addSport(Sport)</code>	<code>boolean</code>	Inserts a sport.
<code>getTotalMemberCount()</code>	<code>int</code>	Aggregate query <code>COUNT(*)</code> .
<code>getTotalChildren()</code>	<code>int</code>	<code>SUM(number_of_children)</code> .
<code>getTotalTeams()</code>	<code>int</code>	<code>SUM(number_of_teams)</code> .
<code>close()</code>	<code>void</code>	Closes the connection.

Design

- Uses **Prepared Statements** to prevent SQL injection.
- The Singleton pattern ensures only one database connection is used throughout the application.
- The `club_sports` table is created but currently not exposed via any public method (exists for future use).

ClubManagementService.java

Service layer that encapsulates all business operations for clubs, members, and sports. It is the single entry point for both the CLI (`Main.java`) and the GUI (`ClubManagementGUI.java`), guaranteeing consistent behaviour regardless of the user interface.

Responsibility

- Translates high-level actions (e.g., “add a club”) into calls on the persistence layer (`DatabaseHandler`).
- Provides the data to the presentation layers in ready-to-use `List` form.
- Shields the UI from direct database access and implementation details.

Initialisation

- Creates a private instance of `DatabaseHandler` via its singleton method.
- The database is initialised automatically the first time `DatabaseHandler.getInstance()` is called.

Public Methods

Club Operations

Method	Returns	Description
<code>addClub(String name, String manager, String location)</code>	<code>boolean</code>	Creates a club with an empty member list and no branches, then persists it.
<code>getAllClubs()</code>	<code>List<Club></code>	Retrieves all clubs (with their branches and members) from the database.

Member Operations

Method	Returns	Description
<code>addMemberToClub(String clubName, int id, String name, String phone, int children)</code>	<code>boolean</code>	Adds a new member to the specified club.
<code>removeMemberFromClub(String clubName, int memberId)</code>	<code>boolean</code>	Removes the member with the given ID from the club.
<code>getAllMembers()</code>	<code>List<Member></code>	Returns all members stored in the database (across all clubs).

Sport Operations

Method	Returns	Description
<code>addSport(String name, int id, int teams)</code>	<code>boolean</code>	Persists a new sport.
<code>getAllSports()</code>	<code>List<Sport></code>	Fetches all sports from the database.

Statistics

Method	Returns	Description
<code>getStatistics()</code>	<code>SystemStatistics</code>	Computes aggregated data: total clubs, members, sports, children, teams, and per-member/sport averages.

The `SystemStatistics` record contains:

- `totalClubs`, `totalMembers`, `totalSports`
- `totalChildren`, `avgChildrenPerMember`
- `totalTeams`, `avgTeamsPerSport`

Design Rationale

- **Single Responsibility:** The service layer owns business logic; the database layer owns persistence. This separation makes the code easier to maintain and test.
- **Shared State:** Both CLI and GUI obtain data through the same `ClubManagementService` instance, so changes made in one interface are immediately visible in the other (uses the same underlying database).
- **Thin Wrapper:** The service mostly delegates calls directly to `DatabaseHandler`. This keeps it simple while still providing a clean API for the UI.

Main.java

Console entry point for the Club Management System. Uses `ClubManagementService` as the single access layer for all data operations, which in turn persists everything in the SQLite database.

Workflow

1. Instantiate the service (`ClubManagementService`) – automatically initialises the database via `DatabaseHandler`.
2. Load clubs, members, and sports from the database **through the service**.
3. Display an interactive menu loop.
4. Execute the chosen operation (display, sort, search, add, remove, statistics).
5. All mutation actions (add club, add member, etc.) call the corresponding service methods, which update the database. After every mutation, the in-memory lists are reloaded from the service to stay synchronised.
6. Sorting algorithms operate on the in-memory lists; they do not modify the database.

Menu Options

Option	Action
1	Display all data (clubs, members, sports).
2	Sort clubs by name using Bubble Sort.
3	Sort members by ID using Selection Sort.
4	Sort sports by name using Merge Sort.
5	Search for a club by name (auto-sorts first).
6	Search for a member by ID (auto-sorts first).
7	Add a new club (persisted to DB via service).
8	Add a new member to a specific club (persisted via service).
9	Add a new sport (persisted to DB via service).
10	Remove a member from a club (persisted to DB via service).
11	Display system statistics (using service-aggregated data).
0	Exit.

Sorting & Searching Flow

- Each sort operation prints the list before and after, along with complexity information.
- Each search automatically performs the required sort (Bubble Sort for clubs, Selection Sort for members) and then performs Binary Search. The execution time is displayed.
- The Merge Sort function returns a new sorted list; the global `sports` reference is updated accordingly.

Design Choices

- All persistence is delegated to `ClubManagementService`, which internally uses the `DatabaseHandler` singleton.
- After every mutating operation, the in-memory lists are refreshed by calling `service.getAllClubs()`, `service.getAllMembers()`, and `service.getAllSports()` to guarantee consistency.
- Statistics are retrieved via the service's `getStatistics()` method, which uses database aggregate functions for efficiency.

- The command-line interface remains completely independent of the GUI, but both share the exact same service layer and database backend.

ClubManagementGUI.java

A Swing-based graphical interface that displays clubs, members, and sports in sortable tables. It uses the same `ClubManagementService` as the CLI, meaning all data is read from (and written to) the SQLite database.

Structure

- **JTabbedPane** with three tabs:
 - **Clubs** – table and controls for Bubble Sort, Binary Search, add club, and member management.
 - **Members** – table with Selection Sort, Binary Search, and refresh.
 - **Sports** – table with Merge Sort and add sport.
- Each tab has a `JTable` backed by a `DefaultTableModel`.
- Data is loaded from the service (**not** from hard-coded sample data). The service fetches the persisted data from the database.

Controls

Tab	Buttons	Action
Clubs	“Add Club”	Opens a dialog; adds a club to the database and refreshes the table.
	“Sort (Bubble)”	Sorts the clubs list in-place using Bubble Sort and refreshes the view.
	“Search Club”	Sorts clubs first, then prompts for a name and displays the result via <code>BinarySearch</code> .
	“Add Member to Club”	Adds a member to the selected club (persisted to database) and refreshes all tables.
	“Remove Member”	Removes a member by ID from the selected club and refreshes.
Members	“Sort by ID”	Sorts all members by ID using Selection Sort, then refreshes the table.
	“Search by ID”	Sorts first, then prompts for an ID and shows the found member.
	“Refresh”	Reloads members from the database via the service.
Sports	“Add Sport”	Adds a new sport (persisted) and refreshes.
	“Sort (Merge)”	Applies Merge Sort to the sports list, updates the view.

Important Notes

- The GUI **now uses** `ClubManagementService` just like the CLI. All add/remove operations are persisted in the SQLite database.
- Sorting is performed on the in-memory lists obtained from the service; no sorting changes are written to the database.
- The Merge Sort returns a new list; the `sports` field is reassigned to the sorted list before refreshing the table.
- A “Show Statistics” button at the bottom opens a dialog with aggregate system statistics, computed via the service.

SortingAlgorithms.java

Provides static sorting methods for clubs, members, sports, and a generic set of algorithms.

Club Sorting – Bubble Sort

```
public static void bubbleSortClubs(List<Club> clubs)
```

- Time: $O(n^2)$, Space: $O(1)$
- Uses `compareToIgnoreCase` for case-insensitive lexicographic order.
- Includes an early-exit optimisation: if no swaps occur in a pass, the list is already sorted.

Member Sorting – Selection Sort

```
public static void selectionSortMembers(List<Member> members)
```

- Time: $O(n^2)$, Space: $O(1)$
- Finds the minimum `id` in the unsorted portion and swaps it with the first unsorted element.
- Always performs exactly $n-1$ swaps (worst-case memory writes are better than Bubble Sort).

Sport Sorting – Merge Sort

```
public static List<Sport> mergeSortSports(List<Sport> sports)
```

- Time: $O(n \log n)$, Space: $O(n)$
- Returns a **new** sorted list (does not mutate the original, though internally works on a copy).
- Stable sort: preserves relative order of equal elements.

Generic Implementations

The class also offers generic methods that accept a `Comparator<? super T>`:

Method	Description
<code>bubbleSort(List<T>, Comparator)</code>	In-place bubble sort.
<code>selectionSort(List<T>, Comparator)</code>	In-place selection sort.
<code>mergeSort(List<T>, Comparator)</code>	Returns a new sorted list, stable.

These can be used for custom sorting beyond the predefined use cases.

BinarySearch.java

Implements both iterative and recursive binary search for clubs and members. Lists **must** be sorted before calling any search method.

Club Search

```
public static Club binarySearchClub(List<Club> clubs, String name)
// Recursive: binarySearchClubRecursive(List<Club>, String)
```

- Searches for a club by name (case-insensitive).
- Time: $O(\log n)$, Space: $O(1)$ iterative / $O(\log n)$ recursive (due to call stack).

Member Search

```
public static Member binarySearchMember(List<Member> members, int id)
// Recursive: binarySearchMemberRecursive(List<Member>, int)
```

- Searches for a member by integer id.
- Time: $O(\log n)$, Space: $O(1)$ / $O(\log n)$ recursive.

Usage Precondition

In the application, the appropriate sorting algorithm is called automatically before a binary search:

- Club search → `bubbleSortClubs(clubs)`
- Member search → `selectionSortMembers(members)`

This ensures the prerequisite is always met.

Algorithm Complexity Analysis

Sorting Algorithms

Bubble Sort – Sorting Clubs by Name

- **Method:** `SortingAlgorithms.bubbleSortClubs(List<Club> clubs)`
- **Use case:** Sorts the list of clubs alphabetically (case-insensitive) by their names.
- **Algorithm description:** Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The largest unsorted element “bubbles up” to its correct position after each pass. An optimisation stops early if a full pass makes no swaps, indicating the list is already sorted.
- **Time Complexity:**
 - **Best case:** $O(n)$ – when the list is already sorted. The early-exit flag (`swapped == false`) causes only one pass over n elements.
 - **Average case:** $O(n^2)$ – on random data, about $\frac{n(n-1)}{4}$ comparisons and swaps are needed.
 - **Worst case:** $O(n^2)$ – when the list is sorted in reverse order, requiring $\frac{n(n-1)}{2}$ comparisons and swaps.
- **Space Complexity:** $O(1)$ The algorithm operates in-place, only using a few temporary variables.
- **Justification:** Bubble Sort was used because clubs are a relatively small dataset and the simplicity of the algorithm serves to illustrate a basic $O(n^2)$ sort. The in-place, stable nature is not critical here, but the early-exit optimisation helps when the list is nearly sorted.

Selection Sort – Sorting Members by ID

- **Method:** `SortingAlgorithms.selectionSortMembers(List<Member> members)`
- **Use case:** Sorts the global member list by their numeric ID.
- **Algorithm description:** Divides the list into a sorted prefix and an unsorted suffix. In each iteration, it scans the unsorted portion to find the smallest element and swaps it with the first element of the unsorted part.
- **Time Complexity:**
 - **Best, average, and worst case:** $O(n^2)$ The number of comparisons is always $\frac{n(n-1)}{2}$, regardless of the initial order. Swaps are at most n .
- **Space Complexity:** $O(1)$ Sorting is performed in-place; only a few index variables are needed.
- **Justification:** Selection Sort is another $O(n^2)$ algorithm that contrasts with Bubble Sort by performing fewer writes (at most $n - 1$ swaps). It was assigned to members because the member list may not benefit from an early exit, and the constant overhead remains low for the expected data sizes.

Merge Sort – Sorting Sports by Name

- **Method:** `SortingAlgorithms.mergeSortSports(List<Sport> sports)`
- **Use case:** Sorts the sports list by name (case-insensitive).
- **Algorithm description:** A classic divide-and-conquer algorithm. It recursively splits the list into two halves until each sub-list contains one element, then merges the sorted halves back together. The merge step uses temporary arrays to hold the halves.
- **Time Complexity:**
 - **Best, average, and worst case:** $O(n \log n)$ The recurrence $T(n) = 2T(n/2) + \Theta(n)$ yields this complexity. The number of levels in the recursion tree is $\log_2 n$, and each level performs a linear merge.
- **Space Complexity:** $O(n)$ Merge Sort requires additional space for the temporary left and right sub-arrays in the merge step. The recursive call stack adds $O(\log n)$ space, but the dominant term is the auxiliary array of total size n .
- **Justification:** Merge Sort was chosen to demonstrate the superior asymptotic performance of $O(n \log n)$ sorting. It is particularly suitable for sorting sports because the number of sports is expected to be moderate, and the guaranteed $O(n \log n)$ time makes it efficient and predictable regardless of the initial ordering. The extra space cost is acceptable.

Searching Algorithm – Binary Search

- **Methods:**
 - `BinarySearch.binarySearchClub(List<Club> clubs, String name)` – iterative
 - `BinarySearch.binarySearchMember(List<Member> members, int id)` – iterative
 - Recursive variants also implemented.
- **Algorithm description:** Binary search works on a **sorted** list. It repeatedly compares the target with the middle element, eliminating half of the remaining search space each time. The iterative version uses a loop while the recursive version calls itself on the appropriate sub-range.
- **Time Complexity:**
 - **All cases (once sorted):** $O(\log n)$ The search space is halved at each step, so the maximum number of comparisons is $\lceil \log_2 n \rceil$.
- **Space Complexity:**
 - **Iterative:** $O(1)$ – only a few variables.
 - **Recursive:** $O(\log n)$ due to the call stack (though tail recursion can be optimised, Java standard compilation does not guarantee it).
- **Prerequisite:** The list must be sorted. In the application, clubs are sorted by name using Bubble Sort and members by ID using Selection Sort before binary search is invoked. This ensures correctness and maintains the $O(\log n)$ search time.

Complexity Summary Table

Algorithm	Used for	Time (Best)	Time (Average)	Time (Worst)	Space
Bubble Sort	Clubs (by name)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	Members (by ID)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	Sports (by name)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Binary Search	Club / Member lookup	-	$O(\log n)$	$O(\log n)$	$O(1)$ iterative / $O(\log n)$ recursive

Build System

The project uses **Gradle** with the **Kotlin DSL** (`build.gradle.kts`). Key choices and rationale:

Plugins

- `java` – standard Java compilation.
- `application` – defines the main class for the CLI.
- `com.gradleup.shadow (v9.4.1)` – creates fat JARs with all dependencies.

Main Classes

- **CLI:** `Main` (set via `application.mainClass`).
- **GUI:** `ClubManagementGUI` (run with custom task `runGUI`).

Dependency

- `org.xerial:sqlite-jdbc:3.51.0.0` – SQLite JDBC driver. Only runtime dependency needed.

Source Sets

- Java sources are located directly under `src/` (not `src/main/java`), configured by `sourceSets.main.java.setSrcDirs(["src"])`.
- Resources (including SQL files in `src/db/`) are also served from `src/`.

Custom Tasks

1. `run` – Runs the CLI with `System.in` connected (for interactive input).
2. `runGUI` – Runs the GUI with `mainClass.set("ClubManagementGUI")`.
3. `shadowJar` – Builds a fat JAR named `Project-cli.jar` with `Main` as entry point.
4. `shadowJarGUI` – Builds a fat JAR named `Project-gui.jar` with `ClubManagementGUI` as entry point.

Why Kotlin DSL?

- Type-safe build scripts, better IDE support (auto-completion, refactoring).
- More concise than Groovy DSL.
- The team can still use Java entirely; the build file is the only Kotlin part.

Nix Shell (shell.nix)

- Provides a reproducible development environment with JDK 21 and Gradle.
- Sets `JAVA_HOME` and makes helper scripts executable.
- Optional; not required for standard builds.